Practical leakage attack on HAWK's Gaussian sampler

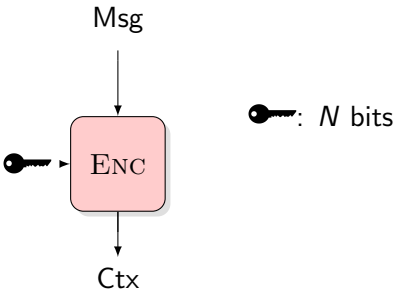
Calvin HAIDAR Clément HOFFMANN

Symposium on Cryptography and Information Security (SCIS)

Monday, January 26, 2026

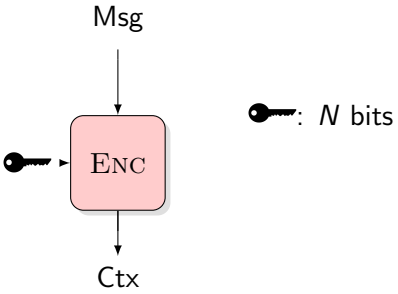


Side-channel attacks



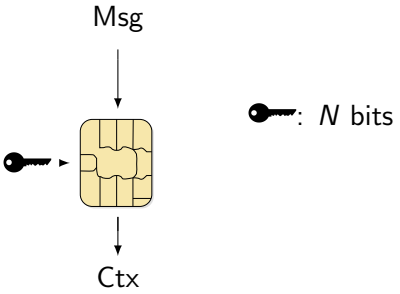
Side-channel attacks

"Cryptographic algorithms don't run on paper,



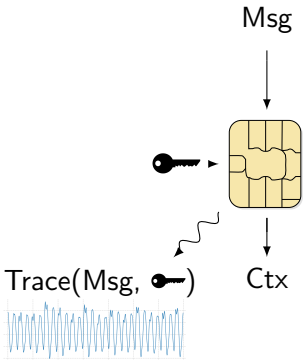
Side-channel attacks

"Cryptographic algorithms don't run on paper, they run on physical devices"



Side-channel attacks

"Cryptographic algorithms don't run on paper, they run on physical devices"

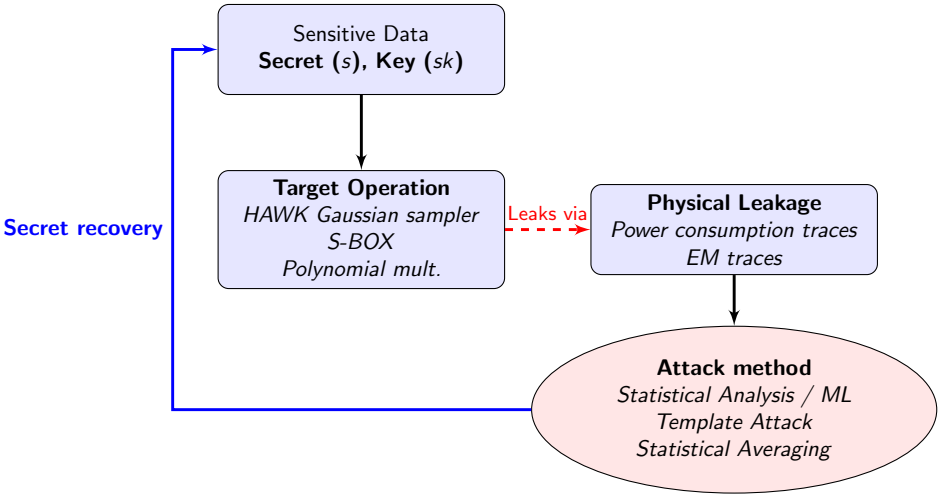


: N bits

For every intermediate value v, an adversary gets a noisy image of the leakage function $L_g(v)$.

Trace : power, EM, acoustics, runtime, ...

From Leakage to Attack: General Principle



HAWK: High-Speed Lattice Signatures



Core Design

- FALCON-inspired
- Lattice-based (LIP/SVP)
- NIST PQC Candidate

Key Benefits

- Efficient signing
- Small signatures
- Compact public keys
- No floating-point arithmetic

"High performance without complex floating-point units"

Focus on Signing: The Gaussian Sampler



To generate a signature, HAWK must sample a vector from a discrete Gaussian distribution.

1. **Pre-computation:** Compute a binary target from the secret key $\mathbf{B}$ and message hash $\mathbf{h}$: $\mathbf{t} = \mathbf{Bh} \pmod 2$.
2. **Gaussian Sampling:** Sample a small integer x with `SamplerSign`.
3. Compute $\mathbf{w} = \mathbf{B}^{-1}\mathbf{x}$. If $\texttt{symbreak}(w) = 0$ then $\bar{\boldsymbol{w}} = -w$. The signature $\mathbf{s} = \frac{1}{2}(\mathbf{h} - \bar{\mathbf{w}})$.

The Vulnerability

We target the sign of coefficients of x in `SamplerSign`.
We handle the `symbreak` in a second time.

SamplerSign



Objective: Sample a vector $\mathbf{x} = (x_0, x_1) \in \mathbb{Z}^{2n}$ from a discrete Gaussian distribution.

Discrete Gaussian Sampling

- The absolute value of each coefficient of x is sampled independently.
- The sign of the coefficient is then chosen randomly
- To reject invalid samples, $r = (\pm\mathbf{x}[i])^2$ is computed.

SamplerSign



Objective: Sample a vector $\mathbf{x} = (x_0, x_1) \in \mathbb{Z}^{2n}$ from a discrete Gaussian distribution.

Discrete Gaussian Sampling

- The absolute value of each coefficient of x is sampled independently.
- The sign of the coefficient is then chosen randomly
- To reject invalid samples, $r = (\pm \mathbf{x}[i])^2$ is computed.

Our target: the sign of the value squared

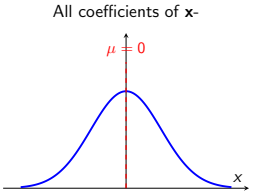
We leverage the fact that the Hamming weight of small negative values is significantly higher than the one of small positive values due to the Two's Complement representation.

In practice, we achieve high-confidence recovery of the signs for all $2n$ coefficients of the sampled vector $\mathbf{x} = (\mathbf{x_0}, \mathbf{x_1})$.

Visualizing Sign Normalization & Averaging



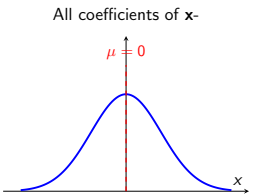
Initial Distributions (x)



- Signs are random.
- Average of each coefficient is zero.
- **Effect:** $\mathbb{E}(\mathbf{x}) = \mathbf{0}$.

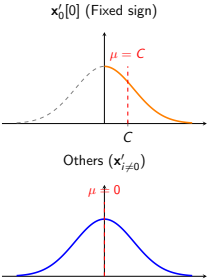
Visualizing Sign Normalization & Averaging

Initial Distributions (x)



- Signs are random.
- Average of each coefficient is zero.
- **Effect:** $\mathbb{E}(\mathbf{x}) = \mathbf{0}$.

Normalized Distributions (x′)



- $\mathbf{x}'_0[0]$ becomes a **Half-Gaussian**.
- Its mean C "leaks" the key $\mathbf{f}$.
- **Effect:** $\mathbb{E}(\mathbf{w}') \approx C \cdot \mathbf{f}$.

The 4 Stages of the Statistical Attack



Once signs are recovered via SCA, we reconstruct the secret key $\mathbf{f}$ using an averaging strategy:

1. **Sign Normalization (Centralization)**
   The sign of $\mathbf{x}$ is random. Knowing the signs, we normalize it into $\mathbf{x}' = \mathbf{x} \cdot (-1)^b$ such that the first coefficient $\mathbf{x}'_0[0]$ is always positive.

The 4 Stages of the Statistical Attack



Once signs are recovered via SCA, we reconstruct the secret key $\mathbf{f}$ using an averaging strategy:

1. **Sign Normalization (Centralization)**
   The sign of $\mathbf{x}$ is random. Knowing the signs, we normalize it into $\mathbf{x}' = \mathbf{x} \cdot (-1)^b$ such that the first coefficient $\mathbf{x}'_0[0]$ is always positive.
2. **Averaging signatures**
   We transform the m signatures $\mathbf{w}_i$ to match the normalized vectors: $\mathbf{w}'_i = \mathbf{f}\mathbf{x}'_{i,0} - \mathbf{g}\mathbf{x}'_{i,1}$

The 4 Stages of the Statistical Attack



Once signs are recovered via SCA, we reconstruct the secret key $\mathbf{f}$ using an averaging strategy:

1. **Sign Normalization (Centralization)**
   The sign of $\mathbf{x}$ is random. Knowing the signs, we normalize it into $\mathbf{x}' = \mathbf{x} \cdot (-1)^b$ such that the first coefficient $\mathbf{x}'_0[0]$ is always positive.
2. **Averaging signatures**
   We transform the m signatures $\mathbf{w}_i$ to match the normalized vectors: $\mathbf{w}'_i = \mathbf{f}\mathbf{x}'_{i,0} - \mathbf{g}\mathbf{x}'_{i,1}$
3. **Key Isolation**
   The expectation $\mathbb{E}(\mathbf{w}')$ isolates $\mathbf{f}$:

$$\mathbb{E}(\mathbf{w}') = \mathbf{f} \cdot \underbrace{\mathbb{E}(\mathbf{x}'_0)}_{\to \mathbb{E}(\mathbf{x}'_0[0])} - \mathbf{g} \cdot \underbrace{\mathbb{E}(\mathbf{x}'_1)}_{\to 0} = C \cdot \mathbf{f}$$

The 4 Stages of the Statistical Attack



Once signs are recovered via SCA, we reconstruct the secret key $\mathbf{f}$ using an averaging strategy:

1. **Sign Normalization (Centralization)**
   The sign of $\mathbf{x}$ is random. Knowing the signs, we normalize it into $\mathbf{x}' = \mathbf{x} \cdot (-1)^b$ such that the first coefficient $\mathbf{x}'_0[0]$ is always positive.
2. **Averaging signatures**
   We transform the m signatures $\mathbf{w}_i$ to match the normalized vectors: $\mathbf{w}'_i = \mathbf{f}\mathbf{x}'_{i,0} - \mathbf{g}\mathbf{x}'_{i,1}$
3. **Key Isolation**
   The expectation $\mathbb{E}(\mathbf{w}')$ isolates $\mathbf{f}$:

$$\mathbb{E}(\mathbf{w}') = \mathbf{f} \cdot \underbrace{\mathbb{E}(\mathbf{x}'_0)}_{\to \mathbb{E}(\mathbf{x}'_0[0])} - \mathbf{g} \cdot \underbrace{\mathbb{E}(\mathbf{x}'_1)}_{\to 0} = C \cdot \mathbf{f}$$

4. **Key Recovery**
   We recover the secret polynomial $\mathbf{f}$ using the empirical average:

$$\mathbf{f} \approx \frac{1}{C \cdot m} \sum_{i=0}^{m-1} \mathbf{w}'_i$$

Handling the `symbreak`



The `symbreak` operation introduces a global sign flip, making $\mathbf{f}$ and $-\mathbf{f}$ indistinguishable during averaging. We resolve this in three steps:

1. **Batch Generation**
   From one physical signature $\bar{\mathbf{w}}$, we generate n related signatures: $(\bar{\mathbf{w}})_i = X^i \cdot \bar{\mathbf{w}}$. **Crucial property:** All signatures in this batch share the *same* `symbreak` value.

Handling the symbreak



The symbreak operation introduces a global sign flip, making $\mathbf{f}$ and $-\mathbf{f}$ indistinguishable during averaging. We resolve this in three steps:

1. **Batch Generation**
   From one physical signature $\bar{\mathbf{w}}$, we generate n related signatures: $(\bar{\mathbf{w}})_i = X^i \cdot \bar{\mathbf{w}}$. **Crucial property:** All signatures in this batch share the *same* symbreak value.
2. **Batch Key Approximation**
   We apply the averaging attack on this specific batch. This yields a rough approximation:

$$\tilde{\mathbf{f}} \approx \pm\mathbf{f}$$

   The global $\pm$ sign is unknown but fixed for the entire batch.

Handling the `symbreak`



The `symbreak` operation introduces a global sign flip, making $\mathbf{f}$ and $-\mathbf{f}$ indistinguishable during averaging. We resolve this in three steps:

1. **Batch Generation**
   From one physical signature $\bar{\mathbf{w}}$, we generate n related signatures: $(\bar{\mathbf{w}})_i = X^i \cdot \bar{\mathbf{w}}$. **Crucial property:** All signatures in this batch share the *same* `symbreak` value.
2. **Batch Key Approximation**
   We apply the averaging attack on this specific batch. This yields a rough approximation:

   $$\tilde{\mathbf{f}} \approx \pm\mathbf{f}$$

   The global $\pm$ sign is unknown but fixed for the entire batch.
3. **Sign Resolution**
   We compare approximations $\tilde{\mathbf{f}}$ from different batches. We align the signs by ensuring consistency (e.g., checking the sign of the coefficient with the highest norm).

Experimental Results



We evaluated our attack on the official `HAWK-512` reference implementation.

Experimental Setup

- **Target:** ARM Cortex-M4 (STM32F4).
- **Leakage:** Power consumption (Current) measured via ChipsWhisperer Pro.
- **Method:** Profiled Template Attack on the squaring operation x^2.

Experimental Results



We evaluated our attack on the official `HAWK-512` reference implementation.

Experimental Setup

- **Target:** ARM Cortex-M4 (STM32F4).
- **Leakage:** Power consumption (Current) measured via ChipsWhisperer Pro.
- **Method:** Profiled Template Attack on the squaring operation x^2.

- **Sign Recovery**
  - **Profiling:** Detect Point of Interests where the leakage is most significant. Build a statistical model for this leakage.
  - **Classification:** For each trace, we map the leakage to two classes that corresponds to signs $\{+1, -1\}$.

Results



- **Sign Recovery Success:** We achieve **100% accuracy** in sign recovery for individual coefficients with a single trace.
- **Key Recovery Efficiency:**

Target	Traces (m)	Success Rate
Signs of $\mathbf{x}$	1	100%
Full Key $\mathbf{f}$	~ 100	100%

- **Impact:** The `symbreak` protection and sign randomization are insufficient to prevent secret key recovery via side-channel averaging.

A masked implementation of the Gaussian sampler is required. Existing implementations are very inefficient. We have been working on a new method, to be submitted to CRYPTO.